Uber Mobile Ride■Hailing Application – Technical Design Document

> **Version**: 1.0.0

> **Author**: Engineering Team

> **Date**: 2026■11■12

---

1. Summary

Uber is a mobile ride■hailing application built with **Expo React Native**.

The app uses **Clerk** for user authentication, **Stripe** for payment processing, and **Neondatabase serverless** for backend data storage.

Key UI features include map integration, driver/rider interfaces, and a bottom■sheet UI for ride details.

> *Evidence:* Architecture summary in the supplied JSON.

---

2. Scope

| Item | Description | Evidence |

|------|-------------|----------|

| **User■Facing Features** | Sign■in, ride booking, driver listing, payment, ride history | Architecture flows |

| **Backend Services** | Neondatabase serverless, Stripe webhook endpoints (not present in repo) | `@neondatabase/serverless`, `@stripe/stripe-react-native` in `package.json` |

| **Geolocation** | Continuous GPS tracking, route rendering | `expo-location`, `react-native-maps`, `react-native-maps-directions` |

| **Authentication** | Clerk + OAuth (Google) | `@clerk/clerk-expo`, `OAuth` component |

| **State Management** | Zustand | `zustand` in `package.json` |

| **Styling** | Tailwind + NativeWind | `tailwindcss`, `nativewind` in `package.json` |

> *Note:* The design document covers only the client■side application. Server■side API routes are not present in the repository.

---

3. Technology Stack

| Layer | Technology | Purpose | Evidence |

|-------|------------|---------|----------|

| **Framework** | Expo (React Native) | Cross■platform mobile app | `expo` in `package.json` |

| **Navigation** | Expo Router | Declarative routing | `expo-router` in `package.json` |

| **UI Components** | React Native + NativeWind | Native UI + Tailwind styling | `react-native`, `nativewind` |

| **Maps** | `react-native-maps`, `react-native-maps-directions` | Map view & directions | `react-native-maps`, `react-native-maps-directions` |

| **Geolocation** | `expo-location` | GPS tracking | `expo-location` |

| **Authentication** | Clerk (`@clerk/clerk-expo`) + OAuth | User auth | `@clerk/clerk-expo` |

| **Payments** | Stripe (`@stripe/stripe-react-native`) | Card processing | `@stripe/stripe-react-native` |

| **Backend DB** | Neondatabase serverless (`@neondatabase/serverless`) | Data persistence | `@neondatabase/serverless` |

| **State** | Zustand | Global state | `zustand` |

| **Utilities** | `expo-secure-store`, `expo-font`, `expo-constants`, `expo-linking` | Secure storage, fonts, constants, deep linking | `expo-secure-store`, `expo-font`, `expo-constants`, `expo-linking` |

| **Testing** | Jest + Expo Test | Unit & integration tests | `jest`, `jest-expo` |

| **Linting/Formatting** | ESLint, Prettier | Code quality | `eslint`, `prettier` |

| **Type System** | TypeScript | Static typing | `typescript` |

---

4. Repository Structure

```
.
■■ .vscode/
■■ app/
■  ■■ (api)/
■  ■  ■■ (stripe)/
■  ■  ■■ ride/
■  ■■ (auth)/
■  ■■ (root)/
■  ■  ■■ (tabs)/
■■ assets/
■  ■■ fonts/
■  ■■ icons/
■  ■■ images/
■■ components/
■■ constants/
■■ lib/
■■ scripts/
■■ store/
■■ types/
■■ package.json
■■ ... (other config files)
```

> *Evidence:* `repository_tree` field in the supplied evidence.

---

5. Architecture Overview

The application follows a **client■centric** architecture with a **serverless backend**.

Key interactions:

1. **User Authentication** – Clerk handles sign■up/sign■in; OAuth component supports Google login.

2. **Ride Booking** – User selects pickup/drop■off via `GoogleTextInput`; route is rendered on `Map`.

3. **Driver Matching** – App queries Neondatabase for nearby drivers; results displayed with `DriverCard`.

4. **Location Tracking** – Continuous GPS updates via `expo-location`; sent to backend.

5. **Payment** – After ride completion, `Payment` component collects card details and processes via Stripe.

> *Evidence:* Architecture flows list in the supplied JSON.

---

6. Components

| Component | File | Purpose | Dependencies |

|---|---|---|---|

| **CustomButton** | `components/CustomButton.tsx` | Reusable action button | None |

| **DriverCard** | `components/DriverCard.tsx` | Displays driver photo, rating, vehicle | None |

| **GoogleTextInput** | `components/GoogleTextInput.tsx` | Autocomplete address input (Google Places API) | `react-native-google-places-autocomplete` |

| **InputField** | `components/InputField.tsx` | Generic text input with validation | None |

| **Map** | `components/Map.tsx` | Map view with user/driver locations & route | `react-native-maps`, `react-native-maps-directions` |

| **OAuth** | `components/OAuth.tsx` | Handles third■party auth via Clerk | `@clerk/clerk-expo` |

| **Payment** | `components/Payment.tsx` | Stripe card collection & processing | `@stripe/stripe-react-native` |

| **RideCard** | `components/RideCard.tsx` | Shows ride details in history & current ride screens | None |

| **RideLayout** | `components/RideLayout.tsx` | Layout wrapper for ride screens | None |

> *Evidence:* Components list in the supplied JSON.

---

7. Flows

| Flow | Description | Key Components | Evidence |

|---|---|---|---|

| **User Authentication Flow** | Clerk (`@clerk/clerk-expo`) + OAuth component | `OAuth`, `CustomButton` | Architecture flows |

| **Ride Booking Flow** | Pickup/drop■off selection → route display | `GoogleTextInput`, `CustomButton`, `Map` | Architecture flows |

| **Payment Flow** | Post■ride card collection & Stripe processing | `Payment` | Architecture flows |

| **Driver Matching Flow** | Query Neondatabase for nearby drivers → display | `DriverCard`, `@neondatabase/serverless` | Architecture flows |

| **Location Tracking Flow** | Continuous GPS updates → backend | `expo-location`, `Map` | Architecture flows |

---

8. APIs

| API | Method | Endpoint | Purpose | Evidence |

|---|---|---|---|---|

| **Ride Booking** | POST | `/api/ride` | Create a new ride request | *Not detected from static analysis* |

| **Driver Matching** | GET | `/api/ride/nearby` | Retrieve nearby drivers | *Not detected from static analysis* |

| **Stripe Webhook** | POST | `/api/stripe/webhook` | Handle payment events | *Not detected from static analysis* |

> *Note:* No API route files are present in the repository (`app/(api)` is empty). The above endpoints are inferred from the architecture description but not confirmed by static analysis.

---

9. Data

| Table | Columns | Purpose | Evidence |

|-------|---------|---------|----------|

| **Users** | `id`, `email`, `name`, `created_at` | Store user profiles | *Not detected from static analysis* |

| **Drivers** | `id`, `name`, `vehicle`, `rating`, `location` | Store driver info | *Not detected from static analysis* |

| **Rides** | `id`, `user_id`, `driver_id`, `pickup`, `dropoff`, `status`, `fare` | Store ride history | *Not detected from static analysis* |

> *Evidence:* `database_tables` field is empty; table definitions are not present in the repository.

---

10. Dependencies

All runtime dependencies are listed in `package.json`.

Key third■party libraries:

• `@clerk/clerk-expo` – Authentication

• `@stripe/stripe-react-native` – Payments

• `@neondatabase/serverless` – Serverless DB

• `expo-location` – GPS

• `react-native-maps` & `react-native-maps-directions` – Map rendering

• `expo-router` – Routing

• `zustand` – State

• `nativewind` + `tailwindcss` – Styling

> *Evidence:* `dependencies` section of `package.json`.

---

11. Configuration

| Config | Source | Purpose | Evidence |

|--------|--------|---------|----------|

| **Clerk API Key** | Environment variable (e.g., `CLERK_PUBLISHABLE_KEY`) | Auth integration | *Not detected from static analysis* |

| **Stripe Publishable Key** | Environment variable | Payment integration | *Not detected from static analysis* |

| **Neondatabase Connection String** | Environment variable | DB connection | *Not detected from static analysis* |

| **Google Places API Key** | Environment variable | Address autocomplete | *Not detected from static analysis* |

| **Expo Config** | `app.json` / `app.config.js` | App metadata, splash screen, etc. | *Not detected from static analysis* |

> *Evidence:* No `.env` or config files were found in the repository tree.

---

12. Security

| Concern | Mitigation | Evidence |

|---------|------------|----------|

| **Third■party auth** | Use Clerk’s secure SDK; OAuth flows handled by Clerk | `@clerk/clerk-expo` |

| **Payment data** | Stripe SDK handles card data; no card data stored locally | `@stripe/stripe-react-native` |

| **Location privacy** | Request user permission via `expo-location`; data sent over HTTPS | `expo-location` |

| **Secure storage** | Sensitive tokens stored in `expo-secure-store` | `expo-secure-store` |

| **API keys** | Stored in environment variables (not in repo) | *Not detected from static analysis* |

---

13. Deployment

The app is built and run with **Expo CLI**.

Typical commands:

```
npm run start          # Start Metro bundler
npm run android        # Run on Android device/emulator
npm run ios            # Run on iOS device/emulator
npm run web            # Run in web browser
```

> *Evidence:* Scripts section of `package.json`.

Deployment to app stores (Google Play / Apple App Store) would involve:

1. Building a standalone bundle (`expo build:android`, `expo build:ios` or `eas build`).

2. Submitting the bundle to the respective store.

> *Evidence:* Not detected from static analysis.

---

14. Operations

| Task | Tool | Notes |

|------|------|-------|

| **Local Development** | Expo CLI | `npm run start` |

| **Testing** | Jest + Expo Test | `npm run test` |

| **Linting** | ESLint | `npm run lint` |

| **Formatting** | Prettier | Integrated with ESLint |

| **State Persistence** | `expo-secure-store` | For tokens, user preferences |

| **Monitoring** | Not present | *Not detected from static analysis* |

---

15. Risks

| Risk | Impact | Mitigation | Evidence |

|------|--------|------------|----------|

| Third■party authentication dependency | Exposure of user data | Use Clerk’s secure SDK; monitor Clerk status | Architecture risks |

| Payment processing security | PCI compliance issues | Stripe SDK handles PCI; secure config | Architecture risks |

| Location data privacy | User consent & data protection | Request permission; secure storage | Architecture risks |

| Serverless DB reliability | Latency/availability | Use Neondatabase’s SLA; fallback logic | Architecture risks |

| Offline functionality | Poor UX | Implement offline caching (not present) | Architecture risks |

| Dependency updates | Breaking changes | Regular dependency audit | Architecture risks |

> *Evidence:* Risks list in the supplied JSON.

---

16. Evidence Summary

| Category | Evidence Source | Key Findings |

|----------|-----------------|--------------|

| **Code** | `repository_tree`, `package.json`, `imports` | Project layout, dependencies, component imports |

| **Architecture** | Architecture JSON | Summary, components, flows, risks |

| **Configuration** | `package.json` scripts | Build & test commands |

| **Missing Data** | `database_tables`, `routes` | No tables or API routes detected |

| **Unknowns** | *Not detected from static analysis* | API endpoints, environment variables, database schema, deployment specifics |

End of Document